POSITION PAPER — THE TWO-PAGE VERSION (FULL PAPER: 30 PP)

Tell your coding agent to work as an architect first

Let it declare intent, let it show the design, let both pass the gates — then, and only then, let it code.

The idea. Before writing code, the assistant must fill in a short **questionnaire**: what parts the system has, how they talk to each other, what happens when something crashes, what it is *not sure about*. Then a small, boring, reliable program — *not* another AI — checks the answers against a checklist of mistakes that real systems have made in production. Missing or fishy answer → fix the declaration first, then write code.

Why boring is the point. A second AI judging the first can be sweet-talked, just like the first one. The checker is a smoke detector, not a critic: simple rules, no opinions, same answer every time. *Is this message allowed to vanish silently? What stops a crash–restart loop?* If the questionnaire doesn't say, that's a finding.

It knows some things better than you. Safety and legal obligations you may never have heard of — the EU CRA's software bill of materials, for one — are applied as defaults whether or not you ask, and the paperwork falls out as a byproduct.

The pipeline

```
prose spec
  → thin spec? AI QUESTIONS YOU FIRST — and declines rather than guesses
  → AI DECLARES the questionnaire (the "SIM")
  → deterministic CHECKER reviews it (the questions, next page)
       ↑____ blocking findings go back to the AI ____↓
  → iterate until it passes  →  compliance report (byproduct)
  → blueprint (typed messages, state machines, invariants)
  → SECOND boring checker validates the blueprint
       ↑____ blocking findings go back ____↓
  → code that mirrors the blueprint one-to-one
  → existing code scanners check the code (they already exist)
  → every decision logged to the project diary
```

The checker never writes code and never blinks; the AI never gets past it with an unanswered question. The checklist itself grows: every real failure the system digests can become a new question — and an AI scout can mine the world's postmortems for candidate questions, but a human approves each one and it enters only as ordinary boring code. The scout never gets a vote at the gate. And the same machinery later *re-reads the finished code* and diffs it against the approved blueprint — so the declaration stays true after the first fifty edits, not just on day one.

The vocabulary (that's all of it)

We say	We mean
SIM	the questionnaire the AI fills out before coding (a structured file)
gate / reviewer	the boring checker programs — two new ones (questionnaire, blueprint) plus the code scanners every ecosystem already has
heuristics H1–H14	the checklist for the questionnaire — "mistakes real systems have made"
checks F1–F6	the checklist for the blueprint — arithmetic: every case has a row, every promise a place in the code
blocking finding	a missing answer serious enough to stop the work
FAM	the blueprint from the approved questionnaire — the *design*, a.k.a. the *architecture* (same thing); code must match it one-to-one
Loop Zero	the questions go to *you* first when the brief is thin — in your words, batched, defaults on record; no answer to the blocking ones → no build (a refusal is a report, not a sulk)

the agent's "plan"	today's prose todo list — unstructured, unchecked, discarded. Not an IR: the blueprint attempted in prose, nothing a checker can total.
decision memory	the project diary: every design choice and *why*, so you never re-argue it next month

The questions the checker asks (the actual list)

#	The question
H1	What happens on each failure this architecture can actually have — crash, message loss, dependency failure, restart loop?
H2	Every guarantee has a concrete mechanism, not an adjective? ("fault tolerant" is not a mechanism.)
H3	May this important command be lost silently? If delivery can duplicate, are receivers idempotent?
H4	Is recovery bounded — what stops a crash-looping component?
H5	No state with *pending work* waits forever with no timeout? (Blocking dequeue — waiting idle for the next message — is fine; suspending started work deadline-free is not.)
H5e	If an actor's behavior can't be enumerated or isn't reproducible (learned policy, stochastic) — what deterministic guardrails and fallback bound it?
H5f	Are continuous or stochastic outputs (torque, valve %, position size) bounded in range?
H5g	Whichever mailbox structure was chosen — are its failure modes addressed (starvation, lowest-priority latency, channel coherence), and is the choice defended?
H6	Is identity tied to a unique incarnation, not a reusable name/address — and decisions made on a fresh view?
H7	Are safety guarantees verified beyond unit tests (fault injection, monitored invariants, model checks)?
H8	Is each safety invariant enforced independently of the party it constrains?
H9	Is every resource created per request/message bounded — a pool, a cap, an owner?
H10	Is any irreversible operation applied before validation, with no reliable rollback?
H11	Do two enforced invariants conflict under a mitigation operation (drain, failover)? *(manual question for now)*
H12	Is a strong guarantee conditional on something the runtime can't enforce — and is that surfaced to operators?
H13	Can the system be stopped on command — enforced, inherited by children, no single point of failure?
H14	Is every external dependency declared — pinned version, stated purpose, none in safety paths without justification? (The bill of materials falls out as a byproduct.)

The second checklist (the blueprint, F1–F6): F1 every case has a row, timeouts where work is in flight; F2 the behavior description matches the declared properties (no neural net hiding in a table); F3 the blueprint refines the questionnaire, invents nothing; F4 ports and bounds match; F5 every guard names its place in the code; F6 a handler for every message. **The third check** is delegated to the code scanners every ecosystem already has.

What is claimed — and what is not

> **Claimed now (engineering):** this assembly — questions first, declared intent, two deterministic gates (intent, design), existing code scanners as the third, diary — is coherent, buildable, and built. The new part is the two upper gates; code-level checking was already mature, which is precisely the point. A single reference run shows the loop can improve a design in three rounds.
>
> **Not yet claimed (science):** that it *reliably* improves software. That is what the blind A/B cold-test measures: same AI, same real issues, with and without the gates — including whether the questionnaire tells the truth about the code, and whether the gate cries wolf often enough to get disabled. The experiment is designed to be able to fail.

Full paper (30 pp): motivation, the compiler-IR framing, feasibility, the drift problem, Loop Zero, the growing knowledgebase, honest limits.
Stanislav Rumega — 21 July 2026